

Sumário

1	Templates	1
1.1	Código padrão (template)	1
1.2	Código padrão com casos teste (template)	1
1.3	Init script	1
1.4	Compile script	1
2	Busca Binária	1
2.1	Funcao pronta	1
2.2	Busca Binária Padrão	2
3	Soma de Prefixos	2
3.1	Soma de prefixos base	2
3.2	Soma de prefixos Bidirecional(Prob Arvore)	2
4	Two pointers	3
4.1	Two pointers(percorrendo esq=0 e dir=n-1)	3
5	Fluxo e Emparelhamento	3
5.1	Dinic (fluxo máximo)	3
6	Estruturas STL	3
6.1	Resumo geral STL	3
7	Estruturas de Dados	4
7.1	Compressão de Coordenadas	4
7.2	PBDS (Set Indexado)	4
7.3	BIT (Fenwick Tree)	5
7.4	BIT 2D	5
7.5	Union-Find (DSU)	5
7.6	Tabela Esparsa (RMQ em O(1))	5
7.7	Segment Tree	6
7.8	Lazy Segment Tree	6
7.9	Segment Tree dinâmica	7
7.10	Segment Tree 2D dinâmica	7
8	Grafos	8
8.1	Ordenação Topológica	8
8.2	Dijkstra	8
8.3	Bellman-Ford	8
8.4	SPFA	9
8.5	Floyd-Warshall	9
8.6	Caminho Euleriano	9
8.7	Componentes Fortemente Conexas (SCC)	10
8.8	Árvore Geradora Mínima (MST)	10
9	Programação Dinâmica	11
9.1	Edit Distance	11
9.2	Problema da Mochila	11
9.3	Longest Increasing Subsequence (LIS)	11
9.4	Máscara de Bits	12
10	Matemática	12
10.1	Fórmulas	12
10.2	Absoluto	14
10.3	MDC & MMC	14
10.4	Exponenciação Binária	14
10.5	Primos	14
10.6	Crivo de Eratóstenes	14
10.7	Fatoração Prima	14
10.8	Função Totiente de Euler (ϕ)	14
10.9	Problema de Josefo (Josephus)	15
10.10	Teorema Chinês do Resto (CRT)	15
11	Strings	15
11.1	Suffix Array	15
11.2	KMP	16
11.3	Rabin-Karp	16
11.4	Trie	16
11.5	Função Z	17
11.6	Manacher	17
12	Árvores	18
12.1	LCA (Menor Ancestral Comum)	18
13	Geometria	18
13.1	Operações básicas	18
13.2	Fecho convexo	19
14	Algoritmos de Ordenação	19
14.1	Insertion Sort	19
14.2	Merge Sort	19
14.3	Quicksort	19
14.4	Counting Sort	20
15	Problemas Especiais	20
15.1	Torre de Hanoi	20

1 Templates

1.1 Código padrão (template)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = LLONG_MAX; //maximo valor

#define fastio ios::sync_with_stdio(0); cin.tie(0);
#define all(v) (v).begin(),(v).end()

int main() {
    fastio;

    // code

    return 0;
}

// LIMITES:
// int -> 2 * 10^9 (32 bits)
// Long Long -> 9 * 10^18 (64 bits)
```

1.2 Código padrão com casos teste (template)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = LLONG_MAX; //maximo valor

#define fastio ios::sync_with_stdio(0); cin.tie(0);
#define all(v) (v).begin(),(v).end()

void solve() {
    // code
}

int main() {
    fastio

    int tc; cin >> tc;
    while (tc--) solve();

    return 0;
}
```

1.3 Init script

```
for f in {A..}; do
    cat base.cpp >> "$f.cpp"
done
```

1.4 Compile script

```
src="$1"
g++ -O2 -Wall -Wextra -Wsign-compare -Wconversion -std=c++17 -g "$src"
```

2 Busca Binária

2.1 Funcao pronta

```
// Lower_bound faz a busca binária na lista
//ordenada de casas

//retorna o indice do primeiro elemento que é menor
//ou igual a x

sort(casas.rbegin(),casas.rend()); //ordena decrescente
auto it=lower_bound(casas.begin(),casas.end(),valor,
greater<ll>())-v.begin();

//retorna o indice do primeiro elemento que é maior
//ou igual a x
```

```

    auto it = lower_bound(casas.begin(), casas.end(),
        valor) - v.begin();
    //retorna o índice do primeiro elemento que é
    //estritamente maior que x
    auto it = upper_bound(casas.begin(), casas.end(),
        valor) - v.begin();
    //acha a quantidade de valores repetidos do "valor"
    int repetidos= upper_bound()-lower_bound();
    //ou
    //seja um valor qualquer do vetor
    auto it = lower_bound(casas.begin(), casas.end(),
        valor);

    // Subtraindo o iterador inicial, descobrimos o
    // índice desse valor instantaneamente
    int indexedovetor = it - casas.begin();

    //se nao encontrar retorna casas.end()

```

2.2 Busca Binária Padrão

```

/*
 * BUSCA BINÁRIA
 *
 * Complexidade: O(log N)
 *
 * Uso comum:
 * 1. Encontrar um elemento em um vetor ordenado.
 * 2. Busca Binária na Resposta (Bsearch on answer) ->
 *    f(x) é monotônica: F F F F V V V V
 *
 * Atenção com os limites:
 * - Evite overflow no cálculo do meio: use `mid = l +
 *   (r - l) / 2;` em vez de `(l + r) / 2`
 * - Cuidado com loops infinitos: verifique se a
 *   atualização usa `l = mid + 1` ou `r = mid - 1`
 */

int busca_binaria(vector<int>&vet, int fim, int x){
    int ini=0;
    int meio;
    while(ini<=fim){
        meio=ini+ (fim-ini)/2;

        if(vet[meio]==x){
            return meio;
        }else
        {
            if(vet[meio]<x){
                ini=meio+1;
            }
            else
            {
                fim=meio-1;
            }
        }
    }
    return -1;
}

```

3 Soma de Prefixos

3.1 Soma de prefixos base

```

//A Soma de Prefixos calcula a soma entre valores em um
//intervalo l e r .Para somar de L até R, você pega
//o total acumulado até R e joga fora tudo o que
//acumulou antes de L (por isso usamos L - 1).
//O(1)

#include <bits/stdc++.h>
//soma de prefixos
using namespace std;
#define fastio ios::sync_with_stdio(0);cin.tie(0);
using ll = long long;

```

```

int main(){
    fastio;
    ll n=5;
    vector<ll>valor;
    vector<ll>pref(n);
    ll x;
    for(int i=0;i<5;i++){
        cin>>x;
        valor.push_back(x);
    }
    pref[0]=valor[0];
    for(int i=1;i<5;i++){
        //pref[i]=a[0]+a[1]+a[2]+a[3]+...+a[i];
        //pref[i]=pref[i-1]+a[i];
        pref[i]=pref[i-1]+valor[i];
    }

    ll q,l,r;
    cin>>q;
    for(int i=0;i<q;i++){
        cin>>l>>r;
        //L sempre começa do 1 para nao erro de pref[l-1]
        //sum[l,r]= sum[1,r]- sum[1,l-1]
        cout<<"soma entre o intervalo l e r = "<<pref[r]-
            pref[l-1]<<endl;
    }

    return 0;
}

```

3.2 Soma de prefixos Bidirecional(Prob Arvore)

```

#include <bits/stdc++.h>

#define fastio ios::sync_with_stdio(0);cin.tie(0);

//soma de prefixos bidirecional
//preparar a soma de prefixos da esquerda para a
//direita
//preparar a soma de prefixos de cima para baixo
using namespace std;
using ll = long long;

int main(){
    fastio;
    ll n,c,x1,y1,x2,y2;
    char x;
    cin>>n>>c;
    ll mat[n+1][n+1]={0};
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            cin>>x;
            if(x=='*'){
                mat[i][j]=1;
            }
            else
            {
                mat[i][j]=0;
            }
        }
    }

    //preparar a soma de prefixos da esquerda para a
    //direita(utilizando mesma matriz)
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            mat[i][j]+=mat[i][j-1];
        }
    }

    //preparar a soma de prefixos de cima para baixo(
    //utilizando mesma matriz)
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            mat[i][j]+=mat[i-1][j];
        }
    }
}

```

```

    }
}

for(int i=0;i<c;i++){
    cin>>x1>>y1>>x2>>y2;
    //somar o todo/subtrair a parte de cima/
    //subtrair a parte da direita/somar pq tirei
    //duas vezes
    //eu nao coloquei mas é so subtrair todo mundo
    //por 1

    ll total = mat[x2][y2];
    ll cima = mat[x1-1][y2];
    ll esquerda = mat[x2][y1-1];
    ll intersecao = mat[x1-1][y1-1];

    cout << total - cima - esquerda + intersecao<<
        "\n";
}

return 0;
}

```

4 Two pointers

4.1 Two pointers(percorrendo esq=0 e dir=n-1)

```

//Algoritmo tratar esq==dir
//questao fala sobre dois valores(seria equivalente ao
//esq )e (dir))
//deu a ideia de um cont sequencial ao longo do for

sort(all(child));
int dir=n-1,esq=0,min=0;

while(esq<=dir){
    if(esq==dir){
        min++;
        break;
    }
    cont+=child[esq]+child[dir];
    if(cont<=x){
        min++;
        esq++;
        dir--;
        cont=0;
    }else
    {
        min++;
        dir--;
        cont=0;
        //
    }
}
cout<<min<<endl;

```

5 Fluxo e Emparelhamento

5.1 Dinic (fluxo máximo)

```

//  $O(V^2 * E)$  geral,  $O(E * \sqrt{V})$  para matching
//bipartido
struct Dinic {
    struct Edge {
        int to, rev;
        ll cap, flow;
    };

    int n;
    vector<vector<Edge>> g;
    vector<int> level, ptr;

    Dinic(int n) : n(n), g(n), level(n), ptr(n) {}

    void add_edge(int u, int v, ll cap) {
        g[u].push_back({v, (int)g[v].size(), cap, 0});

```

```

        g[v].push_back({u, (int)g[u].size()-1, 0, 0});
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);

        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : g[u]) {
                if (level[e.to] == -1 && e.flow < e.cap) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    ll dfs(int u, int t, ll f) {
        if (u == t || f == 0) return f;

        for (int &i = ptr[u]; i < g[u].size(); i++) {
            Edge &e = g[u][i];
            if (level[e.to] == level[u] + 1) {
                ll pushed = dfs(e.to, t, min(f, e.cap -
                    e.flow));
                if (pushed > 0) {
                    e.flow += pushed;
                    g[e.to][e.rev].flow -= pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }

    ll max_flow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(ptr.begin(), ptr.end(), 0);
            while (ll pushed = dfs(s, t, 1e18))
                flow += pushed;
        }
        return flow;
    }

    // Arestas do corte mínimo: visitáveis de s no
    // grafo residual
    vector<pair<int,int>> min_cut(int s) {
        vector<bool> vis(n);
        queue<int> q;
        q.push(s); vis[s] = true;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : g[u])
                if (!vis[e.to] && e.flow < e.cap) {
                    vis[e.to] = true;
                    q.push(e.to);
                }
        }
        vector<pair<int,int>> cut;
        for (int u = 0; u < n; u++)
            if (vis[u])
                for (auto &e : g[u])
                    if (!vis[e.to] && e.cap > 0)
                        cut.push_back({u, e.to});

        return cut;
    }
};

```

6 Estruturas STL

6.1 Resumo geral STL

```

#include <iostream>
#include <vector>
#include <stack>
#include <queue>
#include <utility> // Para pair

```

```

#include <map>
#include <set>

using namespace std;

int main() {
    // =====
    // VECTOR (Array dinâmico, tamanho flexível)
    // O(1) acesso, O(1) no final, O(N) no meio
    // =====
    vector<int> v;

    v.push_back(10); // Adiciona no final [10]
    v.push_back(20); // Adiciona no final [10, 20]
    v.pop_back(); // Remove o último [10]
    int tam = v.size(); // Tamanho atual
    bool v_vazio = v.empty(); // Verifica se está vazio
    v.clear(); // Apaga tudo
    v.erase(v.begin()+1); // remove a posicao 1
    v.resize(5); // reduz ou aumenta o tam do vetor
    v.resize(5,-1); // reduz ou aumenta e os vazios
    // começam com -1, ou seja, valores= (3, 5, 2, -1, -1)
    vector<int> v2(5, -1); // Cria vector de tamanho 5
    // preenchido com -1

    // =====
    // PAIR (Par de dois valores de qualquer tipo)
    // Útil para coordenadas (x, y) ou grafos com peso
    // =====
    vector<pair<int,int>> vs;
    vs.push_back({1,2});
    pair<int, string> p;
    p = make_pair(1, "Filipe"); // Cria o par
    p = {2, "Git"}; // Sintaxe moderna (C++11)
    int id = p.first; // Acessa o 1º elemento (2)
    string nome = p.second; // Acessa o 2º elemento ("Git")

    // =====
    // STACK (Pilha - LIFO: Último a entrar, primeiro a sair)
    // O(1) para inserir, remover e olhar o topo
    // =====
    stack<int> s;
    s.push(5); // Insere o 5 no topo
    s.push(10); // Insere o 10 no topo (Pilha: [5, 10 < topo])
    int topo = s.top(); // Olha o elemento do topo (10) sem remover
    s.pop(); // Remove o elemento do topo
    s.size(); s.empty(); // Tamanho e se está vazia

    // =====
    // QUEUE (Fila - FIFO: Primeiro a entrar, primeiro a sair)
    // O(1) para inserir no fim, remover do início e olhar as pontas
    // =====
    queue<int> q;
    q.push(10); // Entra na fila: [10]
    q.push(20); // Entra na fila: [10, 20]
    int frente = q.front(); // Olha quem está na frente (10)
    int tras = q.back(); // Olha quem está no fim (20)
    q.pop(); // Remove quem está na frente (10 sai, 20 vira a frente)
    q.size(); q.empty(); // Tamanho e se está vazia

    // =====
    // SET (Conjunto ordenado, ELEMENTOS ÚNICOS)
    // Implementado como Árvore. O(Log N) para tudo.
    // =====
    set<int> st;
    st.insert(5); // Insere 5
    st.insert(2); // Insere 2. Conjunto agora: {2, 5} (Sempre ordenado)
    st.insert(5); // Tenta inserir 5 de novo (

```

ignorado, não aceita repetidos)

```

// Buscar elemento: retorna iterator para o item,
// ou st.end() se não achar
if (st.find(2) != st.end()) { /* achou o 2 */ }

// Contar ocorrências: como não repete, serve para
// checar existência (1 ou 0)
if (st.count(5)) { /* 5 existe no set */ }

st.erase(2); // Remove o elemento 2
st.size(); st.empty(); // Tamanho e se está vazio

// =====
// MAP (Dicionário/Tabela - Chave -> Valor)
// Chaves ordenadas e ÚNICAS. O(log N) para
// operações.
// =====
map<string, int> mp;
mp["idade"] = 25; // Cria a chave "idade" com
// o valor 25
mp["pontos"] = 100; // Cria a chave "pontos" com
// o valor 100
mp["idade"] = 26; // Atualiza o valor da chave
// "idade" para 26

int pts = mp["pontos"]; // Acessa o valor (100)

// Buscar chave: funciona igual ao set
if (mp.find("idade") != mp.end()) { /* chave existe
*/ }

mp.erase("pontos"); // Remove a chave "pontos" e
// seu valor
mp.size(); mp.empty(); // Tamanho (quantidade de
// pares) e se está vazio

return 0;
}

```

7 Estruturas de Dados

7.1 Compressão de Coordenadas

// build: $O(N \log N)$ | get: $O(\log N)$

```

// === 1D ===
struct Compress {
    vector<int> c;
    void add(int x) { c.push_back(x); }
    void build() {
        sort(c.begin(), c.end());
        c.erase(unique(c.begin(), c.end()), c.end());
    }
    int get(int x) { return lower_bound(c.begin(), c.end(), x) - c.begin(); }
    int size() { return c.size(); }
};

// === 2D ===
struct Compress2D {
    vector<int> cx, cy;
    void add(int x, int y) { cx.push_back(x); cy.push_back(y); }
    void build() {
        sort(cx.begin(), cx.end());
        cx.erase(unique(cx.begin(), cx.end()), cx.end());
        sort(cy.begin(), cy.end());
        cy.erase(unique(cy.begin(), cy.end()), cy.end());
    }
    int get_x(int x) { return lower_bound(cx.begin(), cx.end(), x) - cx.begin(); }
    int get_y(int y) { return lower_bound(cy.begin(), cy.end(), y) - cy.begin(); }
    int size_x() { return cx.size(); }
    int size_y() { return cy.size(); }
};

```

7.2 PBDS (Set Indexado)

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define ordered_set tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>
using namespace __gnu_pbds;

// ordered_set s;
// s.insert(10); s.insert(20); s.insert(30);
// O(LogN):
// s.find_by_order(1); // Retorna iterador para o 2º (s[i])
// s.order_of_key(30); // Retorna 2 (posição do valor X no set)
```

7.3 BIT (Fenwick Tree)

```
// O(LogN)
struct FenwickTree {
    vector<int> bit;
    int n;

    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    FenwickTree(vector<int> const& v) : FenwickTree(v.size()) {
        for (int i = 0; i < (int)v.size(); i++)
            add(i + 1, v[i]);
    }

    // Soma prefixo [1, r]
    int soma(int r) {
        int ret = 0;
        for (; r > 0; r -= r & (-r))
            ret += bit[r];
        return ret;
    }

    // Soma intervalo [l, r] (1-based)
    int soma(int l, int r) {
        return soma(r) - soma(l - 1);
    }

    // Adiciona delta no índice idx (1-based)
    void add(int idx, int delta) {
        for (; idx <= n; idx += idx & (-idx))
            bit[idx] += delta;
    }
};
```

7.4 BIT 2D

```
// O(LogN * LogM)
struct FenwickTree2D {
    vector<vector<int>> bit;
    int n, m;

    FenwickTree2D(int n, int m) : n(n), m(m), bit(n + 1, vector<int>(m + 1, 0)) {}

    // Adiciona delta em (x, y) (1-based)
    void add(int x, int y, int delta) {
        for (int i = x; i <= n; i += i & (-i))
            for (int j = y; j <= m; j += j & (-j))
                bit[i][j] += delta;
    }

    // Soma prefixo [1,1] até (x, y)
    int soma(int x, int y) {
        int ret = 0;
        for (int i = x; i > 0; i -= i & (-i))
            for (int j = y; j > 0; j -= j & (-j))
                ret += bit[i][j];
        return ret;
    }

    // Soma retângulo (x1,y1) até (x2,y2) (1-based)
    int soma(int x1, int y1, int x2, int y2) {
        return soma(x2, y2)
            - soma(x1 - 1, y2)
            - soma(x2, y1 - 1)
            + soma(x1 - 1, y1 - 1);
    }
};
```

```
}
};
```

7.5 Union-Find (DSU)

```
//  $O\alpha(N) \approx O(1)$ 

vector<int> pai, tam;
void init(int n) {
    pai.resize(n+1);
    tam.assign(n+1, 1);
    iota(pai.begin(), pai.end(), 0); // pai[i] = i
}

int find(int x) {
    if (x == pai[x]) return x;
    return pai[x] = find(pai[x]); // path compression
}

// Union by Size: anexa a árvore menor na maior
void join(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (tam[x] < tam[y]) swap(x, y);

    pai[y] = x;
    tam[x] += tam[y];
}

// Verifica se dois elementos estão no mesmo conjunto
bool mesmoConjunto(int x, int y) {
    return find(x) == find(y);
}

// Retorna o tamanho do componente que contém x
int tamanho(int x) {
    return tam[find(x)];
}
```

7.6 Tabela Esparsa (RMQ em $O(1)$)

```
// Build:  $O(N * \log N)$  | Query:  $O(1)$ 
// Suporta operações idempotentes: min, max, gcd, lcm

vector<int> log2;
vector<vector<int>> st;

void build_sparse(vector<int> &arr) {
    int n = arr.size();
    log2.resize(n + 1);
    log2[1] = 0;
    for (int i = 2; i <= n; i++) log2[i] = log2[i/2] + 1;

    int K = log2[n] + 1;
    st.assign(K, vector<int>(n));
    st[0] = arr;

    for (int j = 1; j < K; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[j][i] = min(st[j-1][i], st[j-1][i + (1 << (j-1))]);
}

// Retorna mínimo no intervalo [L, R] (0-indexed, inclusivo)
int query_min(int L, int R) {
    int j = log2[R - L + 1];
    return min(st[j][L], st[j][R - (1 << j) + 1]);
}

// Versão para array 2D: mínimo em submatriz
// Build:  $O(N * M * \log N * \log M)$  | Query:  $O(1)$ 
vector<vector<vector<vector<int>>>> st2d;

void build_sparse2d(vector<vector<int>> &mat) {
    int n = mat.size(), m = mat[0].size();
    int K = log2[n] + 1, L = log2[m] + 1;
    st2d.assign(K, vector<vector<vector<int>>>>(L, vector<vector<int>>>(n, vector<int>(m))));
}
```

```

for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        st2d[0][0][i][j] = mat[i][j];

for (int j = 1; j < L; j++)
    for (int i = 0; i < n; i++)
        for (int k = 0; k + (1 << j) <= m; k++)
            st2d[0][j][i][k] = min(st2d[0][j-1][i][k],
                                   st2d[0][j-1][i][k + (1 << (j-1))]);

for (int i = 1; i < K; i++)
    for (int j = 0; j < L; j++)
        for (int k = 0; k + (1 << i) <= n; k++)
            for (int l = 0; l + (1 << j) <= m; l++)
                st2d[i][j][k][l] = min(st2d[i-1][j][k][l],
                                       st2d[i-1][j][k + (1 << (i-1))][l]);
}

int query_min2d(int x1, int y1, int x2, int y2) {
    int i = log2[x2 - x1 + 1], j = log2[y2 - y1 + 1];
    return min({st2d[i][j][x1][y1],
                st2d[i][j][x1][y2 - (1 << j) + 1],
                st2d[i][j][x2 - (1 << i) + 1][y1],
                st2d[i][j][x2 - (1 << i) + 1][y2 - (1 << j) + 1]});
}

```

7.7 Segment Tree

```

// O(N) build, O(logN) update e query
//
// Uso:
//   SegTree st(n);
//   for (int i = 1; i <= n; i++) st.vet[i] = valor;
//   st.build(1, 1, n);
//   st.update(1, 1, n, pos, novoValor); //
//   atualiza posição pos
//   st.query(1, 1, n, l, r); // soma
//   em [l, r]
//
// Indexação 1-based. Nó 1 = raiz.
// Filho esquerdo de no = 2*no, direito = 2*no+1
// tree[4*N] garante espaço suficiente para a árvore

struct SegTree {
    vector<ll> vet, tree;
    int n;

    SegTree(int n) : n(n) {
        tree.resize(4*n);
        vet.resize(n + 1);
    }

    // Constrói a árvore a partir de vet[]
    // Chame: build(1, 1, n)
    void build(int no, int l, int r) {
        if (l == r) {
            tree[no] = vet[l];
            return;
        }
        int m = l + (r - l) / 2;
        build(2 * no, l, m);
        build(2 * no + 1, m + 1, r);
        tree[no] = tree[2 * no] + tree[2 * no + 1]; //
        merge (operacao de soma)
    }

    // Atualiza posição pos com valor val (substitui)
    // Chame: update(1, 1, n, pos, val)
    void update(int no, int l, int r, int pos, ll val) {
        if (l == r) {
            tree[no] = val;
            return;
        }
        int m = l + (r - l) / 2;
        if (pos <= m) update(2 * no, l, m, pos, val);
        else update(2 * no + 1, m + 1, r, pos, val);
        tree[no] = tree[2 * no] + tree[2 * no + 1]; //
        atualiza nós pai
    }
}

```

```

}

// Retorna soma em [ql, qr]
// Chame: query(1, 1, n, ql, qr)
ll query(int no, int l, int r, int ql, int qr) {
    if (l > qr || r < ql) return 0; // fora do
    intervalo
    if (l >= ql && r <= qr) return tree[no]; //
    totalmente dentro
    int m = l + (r - l) / 2;

    return query(2 * no, l, m, ql, qr) + query(2 *
        no + 1, m + 1, r, ql, qr);
}
};

```

7.8 Lazy Segment Tree

```

// O(N) build, O(logN) update e query
//
// Lazy: adia updates para filhos até que sejam
// necessários
// Sem lazy, update em intervalo seria O(N * logN)
//
// Uso:
//   SegTreeLazy st(n);
//   for (int i = 1; i <= n; i++) st.vet[i] = valor;
//   st.build(1, 1, n);
//   st.update(1, 1, n, l, r, val); // soma val em todos
//   de [l, r]
//   st.query(1, 1, n, l, r); // soma em [l, r]

struct SegTreeLazy {
    vector<ll> tree, lazy;
    vector<ll> vet;
    int n;

    SegTreeLazy(int n) : n(n) {
        tree.resize(4*n, 0);
        lazy.resize(4*n, 0);
        vet.resize(n + 1, 0);
    }

    void build(int no, int l, int r) {
        if (l == r) { tree[no] = vet[l]; return; }
        int m = l + (r - l) / 2;
        build(2 * no, l, m);
        build(2 * no + 1, m + 1, r);
        tree[no] = tree[2 * no] + tree[2 * no + 1];
    }

    // Desce o lazy do nó atual para seus filhos
    // Chamado antes de qualquer descida na árvore
    void pushDown(int no, int l, int r) {
        if (lazy[no] == 0) return;
        int m = l + (r - l) / 2;
        // Aplica lazy nos filhos
        tree[2 * no] += lazy[no] * (m - l + 1);
        tree[2 * no + 1] += lazy[no] * (r - m);
        lazy[2 * no] += lazy[no];
        lazy[2 * no + 1] += lazy[no];
        lazy[no] = 0; // limpa lazy do nó atual
    }

    // Soma val em todos os elementos de [ql, qr]
    // Chame: update(1, 1, n, ql, qr, val)
    void update(int no, int l, int r, int ql, int qr,
        ll val) {
        if (l > qr || r < ql) return;
        if (l >= ql && r <= qr) {
            // Intervalo totalmente coberto: aplica
            lazy aqui e para
            tree[no] += val * (r - l + 1);
            lazy[no] += val;
            return;
        }
        pushDown(no, l, r); // desce lazy antes de
        dividir
        int m = l + (r - l) / 2;
        update(2 * no, l, m, ql, qr, val);
        update(2 * no + 1, m + 1, r, ql, qr, val);
        tree[no] = tree[2 * no] + tree[2 * no + 1];
    }
}

```



```

    }

    // Soma em [ql, qr]
    // Chame: query(1, 1, n, ql, qr)
    ll query(int no, int l, int r, int ql, int qr) {
        if (l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) return tree[no];
        pushDown(no, l, r); // desce lazy antes de
                             // dividir
        int m = l + (r - l) / 2;
        return query(2 * no, l, m, ql, qr) + query(2 *
            no + 1, m + 1, r, ql, qr);
    }
};

```

7.9 Segment Tree dinâmica

```

// Usa ponteiros em vez de array estático
// O(LogN) por operação, O(Q logN) espaço total (Q =
// número de updates)
//
// Quando usar no lugar da SegTree estática:
// - N muito grande (até 1e18) mas poucas operações
// - Coordenadas não compactadas (ex: queries em [1, 1
// e9])
// - Quer evitar alocar 4*N de antemão
//
// Uso:
// DinamicSegTree st;
// st.update(st.root, 1, N, pos, val);
// st.query(st.root, 1, N, l, r);

struct DinamicSegTree {
    struct Node {
        ll val;
        Node *left, *right;
        Node() : val(0), left(nullptr), right(nullptr)
        {}
    };

    Node* root = new Node();

    void update(Node* no, ll l, ll r, ll pos, ll val) {
        if (l == r) {
            no->val = val;
            return;
        }
        ll m = l + (r - l) / 2;
        // Cria filho só quando necessário (dinâmico)
        if (pos <= m) {
            if (!no->left) no->left = new Node();
            update(no->left, l, m, pos, val);
        } else {
            if (!no->right) no->right = new Node();
            update(no->right, m + 1, r, pos, val);
        }
        // Filho inexistente contribui com 0
        no->val = (no->left ? no->left->val : 0) + (no
            ->right ? no->right->val : 0);
    }

    ll query(Node* no, ll l, ll r, ll ql, ll qr) {
        if (!no || l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) return no->val;
        ll m = l + (r - l) / 2;
        return query(no->left, l, m, ql, qr) + query(no
            ->right, m + 1, r, ql, qr);
    }
};

```

7.10 Segment Tree 2D dinâmica

```

// Segment Tree nas Linhas, outra dinâmica nas colunas
// O(Log^2 N) por operação, O(Q Log^2 N) espaço
//
// Quando usar:
// - Grid N x N com N grande (até 1e9) e poucas
// operações
// - Queries do tipo: soma em retângulo (x1,y1)-(x2,y2)
//
// Uso:

```

```

// SegTree2D st(N); // N = Limite das coordenadas
// st.update(x, y, val);
// st.query(x1, y1, x2, y2);

struct SegTree2D {
    struct NodeY {
        ll val;
        NodeY *left, *right;
        NodeY() : val(0), left(nullptr), right(nullptr)
        {}
    };

    struct NodeX {
        NodeY* yRoot;
        NodeX *left, *right;
        NodeX() : yRoot(new NodeY()), left(nullptr),
            right(nullptr) {}
    };

    NodeX* root;
    ll N;

    SegTree2D(ll N) : N(N), root(new NodeX()) {}

    // Atualiza coluna y com delta na árvore Y do nó X
    void updateY(NodeY* no, ll l, ll r, ll y, ll delta)
    {
        if (l == r) { no->val += delta; return; }
        ll m = l + (r - l) / 2;
        if (y <= m) {
            if (!no->left) no->left = new NodeY();
            updateY(no->left, l, m, y, delta);
        }
        else {
            if (!no->right) no->right = new NodeY();
            updateY(no->right, m + 1, r, y, delta);
        }
        no->val = (no->left ? no->left->val : 0) + (no
            ->right ? no->right->val : 0);
    }

    ll queryY(NodeY* no, ll l, ll r, ll ql, ll qr) {
        if (!no || l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) return no->val;
        ll m = l + (r - l) / 2;
        return queryY(no->left, l, m, ql, qr)
            + queryY(no->right, m + 1, r, ql, qr);
    }

    // Desce na árvore X atualizando a árvore Y de cada
    // nó afetado
    void updateX(NodeX* no, ll l, ll r, ll x, ll y, ll
        delta) {
        updateY(no->yRoot, l, N, y, delta); // atualiza
            este nó X
        if (l == r) return;
        ll m = l + (r - l) / 2;
        if (x <= m) {
            if (!no->left) no->left = new NodeX();
            updateX(no->left, l, m, x, y, delta);
        }
        else {
            if (!no->right) no->right = new NodeX();
            updateX(no->right, m + 1, r, x, y, delta);
        }
    }

    ll queryX(NodeX* no, ll l, ll r, ll x1, ll x2, ll
        y1, ll y2) {
        if (!no || l > x2 || r < x1) return 0;
        if (l >= x1 && r <= x2) return queryY(no->yRoot
            , l, N, y1, y2);
        ll m = l + (r - l) / 2;
        return queryX(no->left, l, m, x1, x2, y1, y2)
            + queryX(no->right, m + 1, r, x1, x2, y1,
            y2);
    }

    // Adiciona delta em (x, y)
    void update(ll x, ll y, ll delta) {
        updateX(root, l, N, x, y, delta);
    }
}

```

```
// Soma em retângulo (x1,y1)-(x2,y2)
ll query(ll x1, ll y1, ll x2, ll y2) {
    return queryX(root, 1, N, x1, x2, y1, y2);
}
};
```

8 Grafos

8.1 Ordenação Topológica

```
// O(V + E)
vector<vector<int>> graph;
vector<int> visited, processing, order;
bool hasCycle = false;

void dfs(int u, int p) {
    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (v == p) continue; // necessario em grafo nao
        // -direcionado;
        if (processing[v]) {
            hasCycle = true;
            return;
        }
        if (!visited[v])
            dfs(v);
    }

    processing[u] = false;
    order.push_back(u);
}

// Detecta e imprime qlqr ciclo encontrado
vector<int> cycle;
bool buscarCiclo(int u) {
    if (processing[u] && cycle.empty()) {
        cycle.push_back(u);
        return cycle.size() == 1;
    }
    if (visited[u]) return false;

    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (buscarCiclo(v)) {
            cycle.push_back(u);
            return cycle.back() != cycle.front();
        }
    }

    processing[u] = false;

    return false;
}

int main() {
    for (int i = 1; i <= n; ++i)
        if (dfs(i)) break; // para de procurar ciclos
        // se achar um

    if (cycle.size()) { // imprime ciclo achado
        cout << cycle.size() << '\n';
        reverse(cycle.begin(), cycle.end());

        for (auto& i : cycle)
            cout << i << ' ';
        cout << '\n';
    }

    return 0;
}
```

8.2 Dijkstra

```
// O((V + E)LogV) ou O(E LogV) - S(V)
const ll N = 2e5 + 1, INF = 1e18;
vector<vector<pair<ll, ll>>> graph;
vector<ll> dist;
```

```
int n, m;

void dijkstra(int s) {
    dist.assign(n + 1, INF);
    priority_queue<pair<ll, ll>> pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (-du > d[u]) continue; // otimização

        // v = vizinho, w = peso
        for (auto &[v, w] : graph[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({-d[v], v}); // -d[v] p manter
                // ordenação crescente
            }
        }
    }
}
```

8.3 Bellman-Ford

```
// Calcula a menor distancia
// entre a e todos os vertices a
// detecta ciclo negativo
// Retorna 1 se ha ciclo negativo
// Nao precisa representar o grafo,
// soh armazenar as arestas
//
// Para achar ciclo positivo basta
// negar todos os pesos, logo
// um ciclo positivo vira negativo
//
// Para caminho máximo:
// Inicializar dist com -INF
// e funcao agora detecta ciclo
// positivo na n-ésima iteração
//
// O(V * E)

const ll INF = 1e18; // 1e9 para int
struct Edge {
    int a, b, c;
};
vector<Edge> arestas;
vector<ll> dist, pai; // pai[i] = no anterior a i
int n, m;

bool bellman_ford(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
    dist[s] = 0;

    int ultimo_relaxado;
    for (int i = 0; i < n; ++i) {
        ultimo_relaxado = -1;
        for (Edge& aresta : arestas) {
            if (dist[aresta.a] < INF && dist[aresta.b]
                > dist[aresta.a] + aresta.c) {
                dist[aresta.b] = dist[aresta.a] +
                    aresta.c;
                pai[aresta.b] = aresta.a;
                ultimo_relaxado = aresta.b;
            }
        }
    }

    return ultimo_relaxado != -1; // true se tem ciclo
    // negativo

// Retorna true se existe qualquer ciclo negativo
bool acharCicloNegativo() {
    dist.assign(n+1, 0);
    pai.assign(n+1, -1);

    ll ciclo;
    for (int i = 0; i < n; ++i) {
```



```

        ciclo = -1;
        for (Edge& a : arestas) {
            if (dist[a.a] < INF && dist[a.b] > dist[a.a
                ] + a.c) {
                dist[a.b] = dist[a.a] + a.c;
                pai[a.b] = a.a;
                ciclo = a.b;
            }
        }
    }

    if (ciclo == -1) return false; // não tem ciclo
    negativo

    for (int i = 0; i < n; ++i) ciclo = pai[ciclo];

    stack<ll> caminho;
    for (ll atual = ciclo;; atual = pai[atual]) {
        caminho.push(atual);
        if (atual == ciclo && caminho.size() > 1)
            break;
    }
    while (!caminho.empty()) {
        cout << caminho.top() << ' ';
        caminho.pop();
    }
    cout << '\n';

    return true;
}

```

8.4 SPFA

```

// Shortest Path Faster Algorithm
// Bellman-Ford com fila – relaxa só vértices que
// mudaram
//  $O(V * E)$  pior caso,  $O(E)$  caso médio na prática
//
// Detecta ciclo negativo contando quantas vezes
// cada vértice entra na fila ( $> N$  vezes = ciclo)
//
// Representação: Lista de adjacência (diferente do
// Bellman-Ford)
// Mais eficiente que Bellman-Ford em grafos esparsos
//
// Para caminho máximo: negar todos os pesos
// Para ciclo positivo: negar todos os pesos e detectar
// ciclo negativo

```

```

const ll INF = 1e18;
vector<pair<int, ll>> adj[MXN]; // adj[u] = {v, peso}
vector<ll> dist, pai;
vector<int> cnt; // quantas vezes cada vértice entrou
// na fila
vector<bool> inQueue;
int n, m;

```

```

// Retorna true se há ciclo negativo alcançável a
// partir de s

```

```

bool spfa(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
    cnt.assign(n + 1, 0);
    inQueue.assign(n + 1, false);

```

```

    dist[s] = 0;
    queue<int> q;
    q.push(s);
    inQueue[s] = true;
    cnt[s] = 1;

```

```

    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pai[v] = u;
                if (!inQueue[v]) {
                    q.push(v);
                    inQueue[v] = true;

```

```

                cnt[v]++;
                // Vértice entrou na fila mais de N
                // vezes → ciclo negativo
                if (cnt[v] > n) return true;
            }
        }
    }
    return false;
}

```

```

// Retorna true se existe qualquer ciclo negativo no
// grafo
// (incluindo componentes não alcançáveis por s)
// Inicializa dist=0 para todos: simula supersource

```

```

bool acharCicloNegativo() {
    dist.assign(n + 1, 0);
    pai.assign(n + 1, -1);
    cnt.assign(n + 1, 0);
    inQueue.assign(n + 1, false);

```

```

    queue<int> q;
    for (int i = 1; i <= n; ++i) {
        q.push(i);
        inQueue[i] = true;
        cnt[i] = 1;
    }

```

```

    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;

```

```

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pai[v] = u;
                if (!inQueue[v]) {
                    q.push(v);
                    inQueue[v] = true;
                    cnt[v]++;
                    if (cnt[v] > n) return true;
                }
            }
        }
    }
    return false;
}

```

8.5 Floyd-Warshall

```

//  $O(V^3)$ ;  $S(V^2)$ 

```

```

int n, m;
void floyd_warshall(vector<vector<ll>>& d) {
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[
                        k][j]);
}

```

```

// no main:

```

```

vector<vector<ll>> d(n+1, vector<ll>(n+1, INF));
for (int i = 1; i <= n; ++i)
    d[i][i] = 0; // dist = 0 para auto-laço

```

8.6 Caminho Euleriano

```

// Algoritmo de Hierholzer
// Para grafos não direcionados:
//  $O(V + E)$ 

```

```

vector<vector<pair<int, int>>> adj; // {vizinho,
// id_aresta}
vector<bool> usado;
int n, m;

```

```

int temCaminhoEuleriano() {
    int ini = -1, impar = 0;

    for (int i = 1; i <= n; ++i) {
        if (!adj[i].empty() && ini == -1) ini = i;

```

```

    if (adj[i].size() & 1)
        impar++, ini = i;
}

if (ini == -1) return 2;
if (impar != 0 && impar != 2) return 0;

vector<int> ptr(n + 1, 0); // próxima aresta a
    visitar
stack<int> st;
vector<int> caminho;
st.push(ini);

while (!st.empty()) {
    int u = st.top();
    // Avança ptr até achar aresta não usada
    while (ptr[u] < (int)adj[u].size() && usado[adj
        [u][ptr[u]].second])
        ptr[u]++;
    if (ptr[u] < (int)adj[u].size()) {
        auto [v, id] = adj[u][ptr[u]++];
        usado[id] = true;
        st.push(v);
    }
    else {
        caminho.push_back(u);
        st.pop();
    }
}

for (int i = 0; i < m; i++)
    if (!usado[i]) return 0;

reverse(caminho.begin(), caminho.end());
for (auto i : caminho) cout << i << ' ';
cout << '\n';

return impar == 0 ? 2 : 1;
}

// Para grafos direcionados:
// O(V + E)
vector<vector<int>> adj;
vector<int> in, out;
int n;
int temCaminhoEuleriano() {
    int ini = -1, fim = -1, temCiclo = 1;

    for (int i = 1; i <= n; i++) {
        if (out[i] - in[i] == 1) {
            if (ini != -1) return 0; // Invalido: soh 1
                no pode ter out > in
            ini = i;
        }
        else if (in[i] - out[i] == 1) {
            if (fim != -1) return 0; // Invalido: soh 1
                no pode ter in > out
            fim = i;
        }
        else if (in[i] != out[i])
            return 0; // Invalido: |in-out| de um no
                NAO pode ser > 1
    }

    // Se não achou ini, é circuito
    if (ini == -1) {
        temCiclo = 2;
        for (int i = 1; i <= n; i++) {
            if (out[i] > 0) {
                ini = i;
                break;
            }
        }
    }

    if (ini == -1) return 0;

    // Algoritmo de Hierholzer
    stack<int> st;
    vector<int> caminho;
    st.push(ini);

```

```

    while (!st.empty()) {
        int u = st.top();
        if (!adj[u].empty()) {
            int v = adj[u].back();
            adj[u].pop_back();
            st.push(v);
        }
        else {
            caminho.push_back(u);
            st.pop();
        }
    }

    // Verifica se o grafo é conexo
    for (int i = 1; i <= n; i++)
        if (!adj[i].empty()) return 0;

    reverse(caminho.begin(), caminho.end());
    for (auto& i : caminho)
        cout << i << ' ';
    cout << '\n';

    return temCiclo; // 1 = Caminho, 2 = Ciclo
}

```

8.7 Componentes Fortemente Conexos (SCC)

```

// O(V + E) - Algoritmo de Kosaraju
// Encontra todos os componentes fortemente conexos em
    grafo direcionado

const int MAXN = 1e5+1;
vector<vector<int>> grafo, grafoReverso;
vector<int> ordem; // Ordem topológica
bitset<MAXN> visitado;
int n, m; // vértices, arestas

// ordenação topológica
void dfsOrdem(int u) {
    visitado[u] = true;
    for (auto& v : grafo[u])
        if (!visitado[v])
            dfsOrdem(v);
    ordem.push_back(u);
}

// DFS no grafo reverso para encontrar SCCs
void dfsComponente(int u) {
    visitado[u] = true;
    for (auto& v : grafoReverso[u])
        if (!visitado[v])
            dfsComponente(v);
}

// Retorna vetor com um representante de cada
    componente
vector<int> kosaraju() {
    for (int i = 1; i <= n; ++i)
        if (!visitado[i])
            dfsOrdem(i);
    reverse(ordem.begin(), ordem.end());

    visitado.reset();
    vector<int> componentes;

    for (auto& u : ordem) {
        if (!visitado[u]) {
            dfsComponente(u);
            componentes.push_back(u);
        }
    }

    return componentes;
}

```

8.8 Árvore Geradora Mínima (MST)

```

// Prim
// O((V + E) * LogV)

const int MAXN = 1e5+1;

```

```

bitset<MAXN> visitado;

int prim() {
    priority_queue<pair<int,int>> pq; // {peso, no}
    pq.emplace(0,1); // começa no no 1

    int sum = 0;
    while (!pq.empty()) {
        auto [w, u] = pq.top();
        pq.pop();

        if (visitado[u]) continue;

        sum += -w;
        visitado[u] = true;

        for (auto& [v,w] : adj[u]) // adj = {no, peso}
            if (!visitado[v])
                pq.emplace(-w, v);
    }

    return visitado[n]? sum : -1;
}

// Kruskal (usa Union-find)
// O(E * logE)
// N precisa guardar grafo, soh arestas
struct Edge {
    int a,b, c;
};
vector<Edge> arestas;
vector<int> p;

int kruskal() {
    for (int i = 1; i <= n; ++i) p[i] = i;
    sort(arestas.begin(), arestas.end(), [](auto a,
        auto b){
            return a.c < b.c;
        });

    int sum = 0;
    for (auto& e : arestas) {
        if (find(e.a) != find(e.b)) {
            join(e.a, e.b);
            sum += e.c;
        }
    }

    return find(1)==find(n)? sum : -1;
}

int find(int x) {
    if (x == p[x]) return x;
    return p[x] = find(p[x]);
}

void join(int x, int y) {
    p[find(x)] = find(y);
}

```

9 Programação Dinâmica

9.1 Edit Distance

```

// Levenshtein Distance
// Problema: menor custo para transformar string s em t
// Operações: inserir (custo 1), remover (custo 1),
//             substituir (custo (s[i]!=t[j]))
// DP 2D: O(N*M) tempo, O(N*M) espaço
int editDistance(string& s, string& t) {
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 1e9));
    dp[0][0] = 0;
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i) dp[i][j] = min(dp[i][j], dp[i-1][j]
                + 1); // remove
            if (j) dp[i][j] = min(dp[i][j], dp[i][j-1]
                + 1); // insere
            if (i && j) dp[i][j] = min(dp[i][j], dp[i-1][j-1]
                + (s[i-1] != t[j-1])); // substitui
        }
    }
}

```

```

    }
    return dp[n][m];
}

// DP 1D: O(N*M) tempo, O(M) espaço
int editDistance1D(string& s, string& t) {
    int n = s.size(), m = t.size();
    vector<int> prev(m+1), cur(m+1);
    iota(prev.begin(), prev.end(), 0);
    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        for (int j = 1; j <= m; j++) {
            cur[j] = min({prev[j] + 1, cur[j-1] + 1,
                prev[j-1] + (s[i-1] != t[j-1])});
        }
        swap(prev, cur);
    }
    return prev[m];
}

```

9.2 Problema da Mochila

```

// Knapsack 0/1 - cada item usado no máximo 1 vez
// dp[j] = maior valor com capacidade exatamente j
// Itera j de trás pra frente: evita usar o item duas
// vezes
// O(N * W)
void knapsack01(int n, int W, vector<ll>& w, vector<ll>
    & v) {
    vector<ll> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int j = W; j >= w[i]; j--)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    cout << dp[W] << '\n';
}

// Knapsack ilimitado - cada item pode ser usado
// infinitas vezes
// Itera j da frente pra trás: permite reusar o mesmo
// item
// O(N * W)
void knapsack(int n, int W, vector<ll>& w, vector<ll>&
    v) {
    vector<ll> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int j = w[i]; j <= W; j++)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    cout << dp[W] << '\n';
}

// Knapsack 0/1 - quando W é muito grande, inverte
// dimensões
// dp[j] = menor peso para atingir exatamente valor j
// Útil quando V (soma dos valores) << W
// O(N * V)
void knapsack01ByValue(int n, int W, vector<ll>& w,
    vector<ll>& v) {
    ll total = 0;
    for (ll x : v) total += x;
    vector<ll> dp(total + 1, LLONG_MAX / 2);
    dp[0] = 0;
    for (int i = 0; i < n; i++)
        for (int j = total; j >= v[i]; j--)
            dp[j] = min(dp[j], dp[j - v[i]] + w[i]);
    ll ans = 0;
    for (ll j = 0; j <= total; j++)
        if (dp[j] <= W) ans = j;
    cout << ans << '\n';
}

```

9.3 Longest Increasing Subsequence (LIS)

```

// O(N * LogN)
int lis(vector<int>& v) {
    int n = v.size();
    vector<int> ans;

    // Initialize the answer vector with the
    // first element of v
    ans.push_back(v[0]);
}

```

```
for (int i = 1; i < n; i++) {
    if (v[i] > ans.back()) {
        ans.push_back(v[i]);
    }
    else {
        int low = lower_bound(ans.begin(), ans.end()
            (), v[i]) - ans.begin();
        ans[low] = v[i];
    }
}

return ans.size();
}
```

9.4 Máscara de Bits

```
// Operações com Bitmask
// Índices 0-based. "bit i" = posição i da direita.
// Uso típico: representar subconjuntos de {0, 1, ..., N-1}

// --- Operações básicas ---
mask | (1 << i) // liga bit i
mask & ~(1 << i) // desliga bit i
mask ^ (1 << i) // inverte bit i
(mask >> i) & 1 // lê bit i (0 ou 1)
mask & (1 << i) // testa bit i (0 se desligado)

// --- Propriedades do conjunto ---
__builtin_popcount(mask) // quantos bits ligados
__builtin_ctz(mask) // índice do bit menos
    significativo ligado
__builtin_clz(mask) // zeros à esquerda (cuidado
    : UB se mask==0)
(mask & (mask-1)) == 0 // mask é potência de 2 (
    exatamente 1 bit)
mask == 0 // conjunto vazio

// --- Iteração ---

// Todos os subconjuntos de {0..N-1}
for (int mask = 0; mask < (1 << n); mask++) { }

// Todos os bits ligados de mask
for (int tmp = mask; tmp; tmp &= tmp-1) {
    int i = __builtin_ctz(tmp); // índice do bit
}

// Todos os subconjuntos de mask (incluindo vazio)
for (int sub = mask; sub > 0; sub = (sub-1) & mask) { }
// Atenção: não itera o subconjunto vazio; adicione
    caso sub==0 se necessário

// --- Máscaras úteis ---
(1 << n) - 1 // todos os n bits ligados
(1 << i) // só o bit i ligado

// --- Padrão DP em subconjuntos ---
// dp[mask][v] = valor considerando exatamente os
    vértices em mask,
// terminando/passando em v
// Transição típica:
// int prev = mask ^ (1 << v); // remove v do
    conjunto
// dp[mask][v] = f(dp[prev][u]) para u vizinho de v
    em prev
//
// Exemplo (Hamiltonian Path - O(2^N * N^2)):
// dp[mask][v] += dp[mask ^ (1<<v)][u] * adj[u][v]
// para todo u em (mask ^ (1<<v)) com adj[u][v] > 0
```

10 Matemática

10.1 Fórmulas

PROGRESSÕES E SOMAS

Soma de consecutivos (PA):
 $S = (\text{primeiro} + \text{último}) * n / 2$

Soma de 1 até N:
 $S = N * (N + 1) / 2$

Soma de quadrados de 1 até N:
 $S = N * (N + 1) * (2N + 1) / 6$

Soma de cubos de 1 até N:
 $S = (N * (N + 1) / 2)^2$

Soma de PG:
 $S = a * (r^n - 1) / (r - 1) \quad (r \neq 1)$

Soma de PG infinita ($|r| < 1$):
 $S = a / (1 - r)$

Termo geral da PA:
 $a_n = a_1 + (n - 1) * d$

Termo geral da PG:
 $a_n = a_1 * r^{(n-1)}$

COMBINATÓRIA

Fatorial:
 $n! = n * (n-1) * \dots * 1$
 $0! = 1$

Combinação:
 $C(n, k) = n! / (k! * (n-k)!)$
 $C(n, 0) = C(n, n) = 1$
 $C(n, k) = C(n-1, k-1) + C(n-1, k)$ (triângulo de Pascal)

Permutação:
 $P(n, k) = n! / (n-k)!$

Permutação com repetição:
 $P = n! / (n_1! * n_2! * \dots * n_k!)$

Arranjo:
 $A(n, k) = n! / (n-k)!$

Número de subconjuntos de N elementos:
 2^N

Princípio da inclusão-exclusão:
 $|A \cup B| = |A| + |B| - |A \cap B|$
 $|A \cup B \cup C| = |A| + |B| + |C| - |nAB| - |nAC| - |nBC| + |nnABC|$

Números de Catalan:
 $C_n = C(2n, n) / (n + 1)$
 $C_0=1, C_1=1, C_2=2, C_3=5, C_4=14, \dots$
 Uso: árvores binárias, parênteses válidos, triangulações

Coefficiente binomial – identidades úteis:
 $C(n, k) = C(n, n-k)$
 $C(n, k)$ para $k=0..n = 2^n$
 $C(n, k)^2$ para $k=0..n = C(2n, n)$

TEORIA DOS NÚMEROS

Divisibilidade – número de divisores:
 Se $n = p_1^{a_1} * p_2^{a_2} * \dots * p_k^{a_k}$
 $d(n) = (a_1+1) * (a_2+1) * \dots * (a_k+1)$

Soma dos divisores:
 $\sigma(n) = \prod (p_i^{a_i+1} - 1) / (p_i - 1)$

Função de Euler (totiente):
 $\phi(n) = n * \prod (1 - 1/p)$ para p primo divisor de n
 $\phi(1) = 1$
 Se p primo: $\phi(p) = p - 1$
 Se p primo: $\phi(p^k) = p^k - p^{(k-1)}$

Pequeno Teorema de Fermat:
 $a^p \equiv a \pmod{p}$ para p primo

$a^{(p-1)} \equiv 1 \pmod{p}$ para p primo e $\text{mdc}(a,p)=1$

Teorema de Euler:

$a\phi^{(n)} \equiv 1 \pmod{n}$ para $\text{mdc}(a,n)=1$

Inverso modular (p primo):

$a^{(-1)} \equiv a^{(p-2)} \pmod{p}$

Teorema Chinês **do** Resto (CRT):

Sistema $x \equiv a_i \pmod{m_i}$ com m_i coprimos dois a dois:

$x = \sum a_i \cdot M_i \cdot y_i \pmod{M}$

onde $M = \prod m_i$, $M_i = M/m_i$, $y_i = M_i^{(-1)} \pmod{m_i}$

MDC / MMC:

$\text{mdc}(a, b) = \text{mdc}(b, a \bmod b)$

$\text{mmc}(a, b) = a * b / \text{mdc}(a, b)$

$\text{mdc}(a, 0) = a$

Identidade de Bézout:

$\text{mdc}(a, b) = a*x + b*y$ (x, y inteiros)

GEOMETRIA

Distância euclidiana:

$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$

Distância Manhattan:

$d = |x_2-x_1| + |y_2-y_1|$

Área **do** triângulo (coordenadas):

$A = |x_1*(y_2-y_3) + x_2*(y_3-y_1) + x_3*(y_1-y_2)| / 2$

Produto vetorial 2D:

$(a \times b) = a.x*b.y - a.y*b.x$

> 0 : b está à esquerda de a (anti-horário)

< 0 : b está à direita de a (horário)

$= 0$: colineares

Produto escalar:

$a \cdot b = a.x*b.x + a.y*b.y = |a||b|\cos\theta()$

Área **do** polígono (Shoelace):

$A = \sum |x_i * y_{i+1} - x_{i+1} * y_i| / 2$

Teorema de Pick (polígono em grade inteira):

$A = I + B/2 - 1$

I = pontos inteiros internos

B = pontos inteiros na borda

A = área (Shoelace)

Círculo:

Área = $\pi * r^2$

Perímetro = $2 * \pi * r$

Setor circular:

Área = $\theta * r^2 / 2$ (θ em radianos)

Fórmula de Herão (triângulo, lados a, b, c):

$s = (a + b + c) / 2$

$A = \sqrt{s * (s-a) * (s-b) * (s-c)}$

TEORIA DOS GRAFOS

Handshaking lemma:

$\text{grau}(v) = 2 * |E|$

Árvore com N vértices:

$|E| = N - 1$

Grafo euleriano (não direcionado):

Circuito: todos os graus pares + conexo

Caminho: exatamente 2 vértices de grau ímpar + conexo

Grafo euleriano (direcionado):

Circuito: $\text{in}[v] = \text{out}[v]$ para todo v + conexo

Caminho: um v com $\text{out}-\text{in}=1$ (início), um com $\text{in}-\text{out}=1$

(fim), resto iguais

Fórmula de Euler (grafo planar):

$V - E + F = 2$ (V vértices, E arestas, F faces incluindo externa)

PROBABILIDADE

Probabilidade condicional:

$P(A|B) = P(A \cap B) / P(B)$

Teorema de Bayes:

$P(A|B) = P(B|A) * P(A) / P(B)$

Valor esperado:

$E[X] = \sum x_i * P(x_i)$

$E[aX + b] = a * E[X] + b$

$E[X + Y] = E[X] + E[Y]$

Variância:

$\text{Var}(X) = E[X^2] - E[X]^2$

EXPONENCIAÇÃO E LOGARITMOS

Exponenciação rápida:

a^n em $O(\log n)$ por repeated squaring

Identidades de log:

$\log(a*b) = \log(a) + \log(b)$

$\log(a/b) = \log(a) - \log(b)$

$\log(a^b) = b * \log(a)$

$\log_b(a) = \log(a) / \log(b)$

Número de dígitos de N na base B :

$d = \text{floor}(\log_B(N)) + 1$

BITMASK

Número de subconjuntos de N elementos:

2^N

Soma de todos os tamanhos de subconjuntos:

$N * 2^{(N-1)}$

STRINGS

Número de substrings de string de tamanho N :

$N * (N + 1) / 2$

MISCELÂNEA

Princípio das casas de pombo (Pigeonhole):

$N+1$ objetos em N caixas $\rightarrow$ alguma caixa tem ≥ 2 objetos

Soma dos primeiros N ímpares:

$1 + 3 + 5 + \dots + (2N-1) = N^2$

Potências de 2:

$2^{10} = 1.024$ ($\sim 10^3$)

$2^{20} = 1.048.576$ ($\sim 10^6$)

$2^{30} = 1.073.741.824$ ($\sim 10^9$)

Limite **int** (32 bits):

$\text{INT_MAX} = 2^{31} - 1 = 2.147.483.647$ ($\sim 2 \cdot 10^9$)

$\text{INT_MIN} = -2^{31} = -2.147.483.648$

Limite **long long** (64 bits):

$\text{LLONG_MAX} = 2^{63} - 1$ ($\sim 9.2 \cdot 10^{18}$)

$\text{LLONG_MIN} = -2^{63}$ ($\sim -9.2 \cdot 10^{18}$)

10.2 Absoluto

```
// abs(x) remove o sinal negativo: retorna o valor puro
// (módulo)
int a = abs(-5); // a vira 5
int b = abs(10); // b continua 10

// CUIDADO EM MARATONA: Para o tipo 'Long Long',
// use sempre llabs()
long long c = llabs(-9223372036854775807LL);
```

10.3 MDC & MMC

```
// O(LogN); n = min(a,b)
// existe a função gcd()
ll mdc(ll a, ll b) {
    if (b == 0) return a;
    return mdc(b, a%b);
}

// existe a função lcm()
ll mmc(ll a, ll b) {
    return (a*b)/mdc(a,b);
}
```

10.4 Exponenciação Binária

```
// a^b O(LogN)
ll bin_pow(ll a, ll b, const ll MOD = 1e9 + 7) {
    a %= MOD;
    ll res = 1;

    while (b) {
        if (b % 2) res = res * a % MOD;
        a = a * a % MOD;
        b /= 2;
    }
    return res;
}

ll inverso(ll a, ll MOD) {
    bin_pow(a, MOD-2); // MOD deve ser primo
}
```

10.5 Primos

```
mashu lucky prime : 91145149
1097774749, 1076767633, 100102021, 999997771
1001010013, 1000512343, 987654361, 999991231
999888733, 98789101, 987777733, 999991921, 1010101333
```

10.6 Crivo de Eratóstenes

```
// O(N * Log(LogN))
vector<bool> eh_primo(TAM, true);
void crivo(int n) {
    eh_primo[0] = eh_primo[1] = false;
    for (int p = 2; p * p <= n; p++) {
        if (eh_primo[p]) {
            for (int i = p * p; i <= n; i += p)
                eh_primo[i] = false;
        }
    }
}
```

10.7 Fatoração Prima

```
// Retorna mapa {primo -> expoente}
// Exemplo: 12 = 2^2 * 3^1 -> {{2,2},{3,1}}

// --- Fatoração de um único número ---
// O(sqrt(N))
map<int,int> fatorar(int n) {
    map<int,int> fat;
    for (int p = 2; p * p <= n; p++) {
        while (n % p == 0) {
            fat[p]++;
            n /= p;
        }
    }
}
```

```
}
// Fator primo restante > sqrt(N)
if (n > 1) fat[n]++;
return fat;
}

// --- Menor fator primo (SPF) ---
// Pré-computa spf[i] = menor fator primo de i
// Pré-processamento: O(N * Log(LogN))
// Fatoração de qualquer n <= N: O(LogN)
int spf[MXN];
void buildSPF(int n) {
    for (int i = 1; i <= n; i++) spf[i] = i;
    for (ll p = 2; p * p <= n; p++) {
        if (spf[p] == p) { // p é primo
            for (int j = p * p; j <= n; j += p)
                if (spf[j] == j) spf[j] = p; // marca menor
                fator
            }
        }
    }
}

map<int,int> fatorarSPF(int n) {
    map<int,int> fat;
    while (n > 1) {
        fat[spf[n]]++;
        n /= spf[n];
    }
    return fat;
}
```

10.8 Função Totiente de Euler (ϕ)

```
//  $\phi(n)$  = quantidade de inteiros em  $[1, n]$  coprimos com n
// Propriedades:
//  $\phi(1) = 1$ 
// p primo  $\rightarrow \phi(p) = p - 1$ 
// p primo  $\rightarrow \phi(p^k) = p^k - p^{k-1}$ 
//  $mdc(a,b)=1 \rightarrow \phi(a*b) = \phi(a) * \phi(b)$  (multiplicativa)
//
// Fórmula:  $\phi(n) = n * \prod (1 - 1/p)$  para todo primo  $p | n$ 
//  $= n * \prod (p-1)/p$ 
// Exemplo:  $\phi(12) = 12 * (1-1/2) * (1-1/3) = 12 * 1/2 * 2/3 = 4$ 
//
// Uso com exponenciação modular:
//  $a^b \bmod c = a^{(b \% \phi(c) + \phi(c))} \bmod c$  se  $b \geq \phi(c)$ 
//  $= a^b \bmod c$  se  $b < \phi(c)$ 

// --- Totiente de um único número ---
// O(sqrt(N))
int phi(int n) {
    int res = n;
    // Fatoramos n e para cada primo p encontrado
    // aplicamos res = res * (p-1)/p
    // (equivalente a res -= res/p, evitando divisão com perda)
    for (int p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            // p é fator primo de n
            res = res / p * (p - 1); // res *= (1 - 1/p)
            while (n % p == 0) n /= p; // remove todas as ocorrências de p
        }
    }
    // Se ainda sobrou um fator > 1, ele é primo
    if (n > 1) res = res / n * (n - 1);
    return res;
}
```

```
// --- Tabela de totientes para  $[1, n]$  ---
// O(N * Log(LogN)) - igual ao crivo de Eratóstenes
// Ideia: para cada primo p, multiplica todos os
// múltiplos por (p-1)/p
int phi_table[MXN];
void phitable(int n) {
    for (int i = 1; i <= n; i++) phi_table[i] = i;
    for (int p = 2; p <= n; p++) {
        if (phi_table[p] == p) { // p é primo
            for (int j = p * p; j <= n; j += p)
                phi_table[j] = phi_table[j] / p * (p - 1);
        }
    }
}
```



```
// Inicializa  $\phi(i) = i$  para todo  $i$  (valor antes de
// aplicar os fatores)
for (int i = 1; i <= n; i++) phi_table[i] = i;

for (int p = 2; p <= n; p++) {
    // Se  $\phi(p)$  ainda é  $p$ , então  $p$  é primo (não foi
    // modificado por nenhum fator)
    if (phi_table[p] == p) {
        // Aplica o fator  $(p-1)/p$  em todos os múltiplos
        // de  $p$ 
        for (int j = p; j <= n; j += p)
            phi_table[j] = phi_table[j] / p * (p - 1);
    }
}
```

10.9 Problema de Josefo (Josephus)

```
// === Josephus Problem ===
//  $n$  pessoas numeradas  $0..n-1$  em círculo, remove  $k$ -
// ésima a cada passo
// Retorna posição do sobrevivente ( $0$ -indexed)
//
// Complexidade:  $O(n)$  ingênuo,  $O(\log n)$  para  $k=2$ ,  $O(k \log n)$  caso geral
//
// Fórmula clássica:  $josephus(n, k) = (josephus(n-1, k) + k) \% n$ 

// ---  $k$  fixo = 2 (recorrência eficiente  $O(\log n)$ ) ---
// Base:  $josephus(1) = 0$ 
//  $josephus(2n) = 2 * josephus(n) + 1$ 
//  $josephus(2n+1) = 2 * josephus(n) - 1$ 
ll josephus2(ll n) {
    if (n == 1) return 0;
    if (n & 1) return 2 * josephus2(n / 2) + 1;
    return 2 * josephus2(n / 2) - 1;
}

// ---  $k$  arbitrário  $O(k \log n)$  ---
ll josephus(ll n, ll k) {
    if (n == 1) return 0;
    if (k == 1) return n - 1;
    if (k > n) return (josephus(n - 1, k) + k) % n;
    ll cnt = n / k;
    ll res = josephus(n - cnt, k);
    res -= n % k;
    if (res < 0) res += n;
    else res += res / (k - 1);
    return res;
}

// --- Versão iterativa  $O(n)$  para  $n$  pequeno ---
ll josephus_iter(ll n, ll k) {
    ll res = 0;
    for (ll i = 2; i <= n; i++)
        res = (res + k) % i;
    return res; //  $0$ -indexed
}
```

10.10 Teorema Chinês do Resto (CRT)

```
// Resolve sistema:  $x \equiv a[i] \pmod{m[i]}$ 
// Complexidade:  $O(N \log M)$  onde  $M = \prod(m[i])$ 
// Retorna  $\{x, M\}$  ou  $\{-1, -1\}$  se sem solução

pair<ll, ll> crt(vector<ll> &a, vector<ll> &m) {
    ll x = 0, M = 1;
    for (int i = 0; i < a.size(); i++) {
        ll ai = a[i] % m[i];
        if (ai < 0) ai += m[i];

        // Resolve:  $M * t \equiv ai - x \pmod{m[i]}$ 
        ll g = gcd(M, m[i]);
        if ((ai - x) % g != 0) return {-1, -1};

        ll M_ = M / g, m_ = m[i] / g;
        ll t = ((ai - x) / g) * inv_mod(M_ % m_, m_) % m_;
        if (t < 0) t += m_;
    }
```

```
x += t * M;
M *= m[i] / g;
x %= M;
}
return {x, M};
}

// Garner's algorithm: recupera  $x \pmod{MOD}$  dado  $x \equiv a[i] \pmod{m[i]}$ 
// Requer  $m[i]$  coprimos entre si
// Complexidade:  $O(N^2)$ 
ll garner(vector<ll> &a, vector<ll> &m, ll MOD) {
    int n = a.size();
    vector<ll> x(n);
    for (int i = 0; i < n; i++) {
        x[i] = a[i];
        for (int j = 0; j < i; j++) {
            x[i] = (x[i] - x[j]) * inv_mod(m[j], m[i]) % m[i];
            if (x[i] < 0) x[i] += m[i];
        }
    }
    ll ans = 0, mult = 1;
    for (int i = 0; i < n; i++) {
        ans = (ans + x[i] * mult) % MOD;
        mult = mult * m[i] % MOD;
    }
    return ans;
}
```

11 Strings

11.1 Suffix Array

```
//  $sa[i]$  = índice do  $i$ -ésimo menor sufixo de  $s$  ( $0$ -based)
//  $rank[i]$  = posição do sufixo  $i$  no SA
//  $lcp[i]$  = longest common prefix entre  $sa[i]$  e  $sa[i-1]$ 
//
// Usos:
// - Busca de padrão: binary search no SA
// - Substring mais longa que aparece  $\geq k$  vezes: maior intervalo de  $LCP \geq k-1$ 
// - Número de substrings distintas:  $n*(n+1)/2 - \sum(lcp)$ 
// - Comparar sufixos rapidamente:  $rank[i] < rank[j] \Rightarrow$  sufixo  $i <$  sufixo  $j$ 
// - Maior substring comum entre strings: concatenar com '#' e checar LCP
//
//  $O(N \log N)$ 
vector<int> build_sa(string s) {
    s += "$";
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);

    // ordena pelos primeiros caracteres
    sort(sa.begin(), sa.end(), [&](int i, int j) {
        return s[i] < s[j]; });

    r[sa[0]] = 0;
    for (int i = 1; i < n; i++)
        r[sa[i]] = r[sa[i-1]] + (s[sa[i]] != s[sa[i-1]]);

    // dobra o tamanho do prefixo comparado
    for (int k = 1; k < n; k <= 1) {
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (r[i] != r[j]) return r[i] < r[j];
            int ri = (i + k < n) ? r[i + k] : -1;
            int rj = (j + k < n) ? r[j + k] : -1;
            return ri < rj;
        });

        nr[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            nr[sa[i]] = nr[sa[i-1]] + ((r[sa[i]] != r[sa[i-1]]) || ((sa[i]+k < n ? r[sa[i]+k] : -1) != (sa[i-1]+k < n ? r[sa[i-1]+k] : -1)));
    }
```

```

    r = nr;
    if (r[sa[n-1]] == n-1) break; // ranks todos
    // distintos
}

return sa; // último elemento é o '$' (n-1)
}

// Kasai's algorithm O(N)
vector<int> build_lcp(string &s, vector<int> &sa) {
    int n = s.size();
    vector<int> rank(n), lcp(n);
    for (int i = 0; i < n; i++) rank[sa[i]] = i;

    int k = 0;
    for (int i = 0; i < n; i++) {
        if (rank[i] == n-1) { k = 0; continue; } //
        // último no SA (o '$')
        int j = sa[rank[i] + 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+
            k]) k++;
        lcp[rank[i]] = k;
        if (k) k--;
    }
    return lcp; // lcp[i] = LCP entre sa[i] e sa[i-1],
    // lcp[0] = 0
}

// Busca padrão p: retorna {primeira, última} posição
// no SA
// s deve ter '$' no final
// Quantidade de p no texto será second - first + 1, se
// existir p (first != -1)
// O(|p| * logN)
pair<int,int> find_pattern(string &s, vector<int> &sa,
    string &p) {
    int n = s.size(), m = p.size();
    int L = 0, R = n-1;

    // Lower_bound
    int first = -1;
    while (L <= R) {
        int mid = (L + R) / 2;
        int cmp = s.compare(sa[mid], m, p);
        if (cmp >= 0) {
            if (cmp == 0) first = mid;
            R = mid - 1;
        }
        else L = mid + 1;
    }
    if (first == -1) return {-1, -1};

    // upper_bound
    L = 0, R = n-1;
    int last = first;
    while (L <= R) {
        int mid = (L + R) / 2;
        int cmp = s.compare(sa[mid], m, p);
        if (cmp <= 0) {
            if (cmp == 0) last = mid;
            L = mid + 1;
        }
        else R = mid - 1;
    }
    return {first, last};
}

```

11.2 KMP

```

// p[i] = comprimento do maior prefixo próprio do
// padrão s que também é sufixo
// do texto t[0..i] Prefixo/Sufixo próprio é um prefixo
// /sufixo que não é a
// string inteira Usos:
// - Busca de padrão único (igual Z, mas mais
// natural para busca online)
// - Busca online/streaming: processa t caractere a
// caractere sem ter t
// completo
// - Menor período de string: n - lps[n-1] é o
// período mínimo
// - Autômato de string: transformar lps em DFA

```

```

    para múltiplas consultas no
    // mesmo padrão

// O(m), m = s.size()
vector<int> pi(string s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); ++i) {
        while (j > 0 && s[j] != s[i]) j = p[j - 1];
        if (s[j] == s[i]) j++;
        p[i] = j;
    }
    return p;
}

// O(n + m) KMP: busca padrão s em texto t, retorna
// índices de cada ocorrência
// (0-based)
vector<int> kmp(string &t, string &s) {
    vector<int> p = pi(s + '$'), match;
    for (int i = 0, j = 0; i < t.size(); ++i) {
        while (j > 0 && s[j] != t[i]) j = p[j - 1];
        if (s[j] == t[i]) j++;
        if (j == s.size()) match.push_back(i - j + 1);
    }
    return match;
}

```

11.3 Rabin-Karp

```

// O(N + M) médio, O(N * M) pior caso
// Busca de padrão p em texto t usando hashing de
// janela deslizante
// Usos:
// - Busca de múltiplos padrões simultaneamente (coloca
// todos num set de hashes)
// - Detectar strings repetidas / substrings duplicadas
// (+busca binária)

const ll MOD = 1e9 + 7, BASE = 31;
vector<int> rabin_karp(string t, string p) {
    int n = t.size(), m = p.size();
    vector<int> ocorrencias;

    // Pré-computa potências de BASE
    vector<ll> pw(max(n, m) + 1);
    pw[0] = 1;
    for (int i = 1; i <= max(n, m); i++) pw[i] = pw[i -
        1] * BASE % MOD;

    // Hash do padrão
    ll hp = 0;
    for (int i = 0; i < m; i++) hp = (hp + (p[i] - 'a'
        + 1) * pw[i]) % MOD;

    // Hash prefixo do texto: h[i] = hash(t[0..i-1])
    vector<ll> h(n + 1, 0);
    for (int i = 0; i < n; i++) h[i + 1] = (h[i] + (t[i]
        - 'a' + 1) * pw[i]) % MOD;

    // Hash de t[l..r-1] = (h[r] - h[l]) / pw[l]
    // = (h[r] - h[l]) * inv(pw[l])
    // Evita inversão modular: compara (h[r] - h[l])
    // com hp * pw[l]
    for (int i = 0; i + m <= n; i++) {
        ll ht = (h[i + m] - h[i] + MOD) % MOD;
        if (ht == hp * pw[i] % MOD) ocorrencias.
            push_back(i);
    }

    return ocorrencias; // índices (0-based) onde p
    // ocorre em t
}

```

11.4 Trie

```

// Armazena strings para buscas eficientes por prefixo
// O(N) por inserção e busca, onde N = tamanho da
// string
// Usos comuns em CP:
// - Contar strings com determinado prefixo

```

```
// - Verificar se string/prefixo existe
// - XOR máximo com Trie binária (ver abaixo)
//
// Como usar:
//   Trie t;
//   t.insert("hello");
//   t.search("hello") → true (string completa existe)
//   t.search("hell") → false (prefixo existe mas
// string não foi inserida)
//   t.startsWith("hell") → true (prefixo existe)
//   t.count("hel") → quantas strings inseridas
// começam com "hel"

struct Trie {
    struct Node {
        int filho[26]; // índice dos filhos (a=0, b=1,
            ..., z=25)
        int fim; // quantas strings terminam
            aqui
        int passa; // quantas strings passam por
            este nó
        Node() : fim(0), passa(0) { fill(filho, filho +
            26, -1); }
    };

    vector<Node> nos;

    Trie() { nos.emplace_back(); } // nó raiz = índice
        0

    void insert(const string& s) {
        int cur = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nos[cur].filho[ch] == -1) {
                nos[cur].filho[ch] = nos.size();
                nos.emplace_back();
            }
            cur = nos[cur].filho[ch];
            nos[cur].passa++;
        }
        nos[cur].fim++;
    }

    // Retorna true se a string completa foi inserida
    bool search(const string& s) {
        int cur = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nos[cur].filho[ch] == -1) return false;
            cur = nos[cur].filho[ch];
        }
        return nos[cur].fim > 0;
    }

    // Retorna true se alguma string inserida começa
    // com s
    bool startsWith(const string& s) {
        int cur = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nos[cur].filho[ch] == -1) return false;
            cur = nos[cur].filho[ch];
        }
        return true;
    }

    // Quantas strings inseridas começam com o prefixo
    // s
    int count(const string& s) {
        int cur = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nos[cur].filho[ch] == -1) return 0;
            cur = nos[cur].filho[ch];
        }
        return nos[cur].passa;
    }
};
```

11.5 Função Z

```
// O(n)
// z[i] = comprimento do maior prefixo de s que também
// é prefixo de s[i..]
// Usos:
//   - Busca de padrão: z(p + "#" + t), ocorrências
//     onde z[i] == |p|
//   - Menor período de string: menor k tal que k | n
//     e z[k] == n - k
//   - Contar prefixos distintos que ocorrem em s:
//     acumular z[i] com cuidado
//   - Comparar sufixos rapidamente sem suffix array

vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;

        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

11.6 Manacher

```
// Encontra o maior palíndromo centrado em cada posição
// O(N) – linear, sem expansão quadrática
//
// Ideia central: reaproveita palíndromos já calculados
// usando a simetria do espelho em torno do centro
// atual
//
// String transformada: intercala '#' entre caracteres
// "abc" → "@#a#b#c#$"
// - '@' e '$' são sentinelas (evitam checagem de
// bounds)
// - '#' entre caracteres unifica palíndromos pares e
// ímpares
// - p[i] = raio do palíndromo centrado em i na
// string transformada
// - palíndromo real de tamanho p[i] (par ou ímpar)
//
// Exemplo: "abacaba"
// t = "@#a#b#a#c#a#b#a#$"
// p = 0 1 0 3 0 1 0 7 0 1 0 3 0 1 0
//
// Como usar:
// 1. Chame manacher(s) para pré-computar p[]
// 2. Use getLongest(i, true) → raio do maior
// palíndromo ímpar centrado em i
// Use getLongest(i, false) → raio do maior
// palíndromo par centrado entre i e i+1
// 3. O tamanho real do palíndromo é igual ao raio
// retornado
// ímpar: tamanho = 2*raio + 1, começa em i -
// raio
// par: tamanho = 2*raio, começa em i -
// raio + 1
//
// Exemplo de uso (maior palíndromo):
// manacher(s);
// for (int i = 0; i < s.size(); i++) {
//     int r = getLongest(i, true); // ímpar
//     // palíndromo: s.substr(i - r, 2*r + 1)
//     int r = getLongest(i, false); // par
//     // palíndromo: s.substr(i - r + 1, 2*r)
// }

vector<int> p;

void manacher(const string& s) {
    string t = "@";
    for (char c : s) t += "#" + string(1, c);
```

```

t += "$";

int n = t.size();
p.assign(n, 0);

int c = 0, r = 0;
for (int i = 1; i < n - 1; i++) {
    int mirror = 2 * c - i;
    if (i < r) p[i] = min(r - i, p[mirror]);
    while (t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
    if (i + p[i] > r) { c = i; r = i + p[i]; }
}

// ímpar: centro real em t[2*i + 2] (o próprio caractere)
// par: centro real em t[2*i + 3] (o '#' entre i e i+1)
int getLongest(int i, bool isOdd) {
    return p[2 * i + 2 + !isOdd];
}

```

12 Árvores

12.1 LCA (Menor Ancestral Comum)

```

// Build: O(N log N) | Query: O(log N)
vector<vector<int>> graph;
vector<vector<int>> up;
vector<int> depth;
int LOG;

void dfs_lca(int u, int p) {
    up[u][0] = p;
    for (int i = 1; i <= LOG; i++)
        up[u][i] = up[up[u][i-1]][i-1];

    for (int v : graph[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfs_lca(v, u, graph);
        }
    }
}

void build_lca(int root = 0) {
    int n = graph.size();
    LOG = 32 - __builtin_clz(n);
    up.assign(n, vector<int>(LOG + 1));
    depth.assign(n, 0);
    dfs_lca(root, root, graph);
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);

    // Sobe u para mesma profundidade de v
    int diff = depth[u] - depth[v];
    for (int i = 0; i <= LOG; i++)
        if (diff & (1 << i)) u = up[u][i];

    if (u == v) return u;

    // Sobe ambos até o LCA
    for (int i = LOG; i >= 0; i--)
        if (up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    return up[u][0];
}

// Distância entre dois vértices
int dist_lca(int u, int v) {
    return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}

// k-ésimo ancestral (k=0: próprio nó, k=1: pai)
int kth_ancestor(int u, int k) {
    for (int i = 0; i <= LOG; i++)
        if (k & (1 << i)) u = up[u][i];
    return u;
}

```

13 Geometria

13.1 Operações básicas

```

// === 2D Geometry Basics ===
// Complexidade: O(1) para todas operações básicas

using ld = long double;
const ld EPS = 1e-9;
const ld PI = acos(-1);

struct Point {
    ld x, y;
    Point(ld x = 0, ld y = 0) : x(x), y(y) {}

    Point operator+(Point p) { return Point(x + p.x, y + p.y); }
    Point operator-(Point p) { return Point(x - p.x, y - p.y); }
    Point operator*(ld d) { return Point(x * d, y * d); }
    Point operator/(ld d) { return Point(x / d, y / d); }

    bool operator<(Point p) const {
        if (fabs(x - p.x) > EPS) return x < p.x;
        return y < p.y - EPS;
    }
    bool operator==(Point p) { return fabs(x - p.x) < EPS && fabs(y - p.y) < EPS; }

    ld dist2() { return x * x + y * y; }
    ld dist() { return sqrt(dist2()); }
    ld dot(Point p) { return x * p.x + y * p.y; }
    ld cross(Point p) { return x * p.y - y * p.x; }
    ld angle() { return atan2(y, x); }
    Point rotate(ld a) { return Point(x*cos(a) - y*sin(a), x*sin(a) + y*cos(a)); }
    Point perp() { return Point(-y, x); } // rotaciona 90° CCW
    Point unit() { return *this / dist(); }
};

// Distância entre pontos
ld dist(Point a, Point b) { return (a - b).dist(); }

// Ângulo entre vetores (0 a PI)
ld angle(Point a, Point b) { return acos(a.dot(b) / (a.dist() * b.dist())); }

// Orientação: 0 = colinear, >0 = CCW, <0 = CW
ld orient(Point a, Point b, Point c) { return (b - a).cross(c - a); }

// Projeção de p na reta ab
Point project(Point a, Point b, Point p) {
    Point v = b - a;
    return a + v * ((p - a).dot(v) / v.dist2());
}

// Distância de ponto p à reta ab
ld dist_to_line(Point a, Point b, Point p) {
    return fabs((p - a).cross(b - a)) / (b - a).dist();
}

// Distância de ponto p ao segmento ab
ld dist_to_seg(Point a, Point b, Point p) {
    if (a == b) return dist(a, p);
    Point v = b - a;
    ld t = (p - a).dot(v) / v.dist2();
    if (t < 0) return dist(a, p);
    if (t > 1) return dist(b, p);
    return dist_to_line(a, b, p);
}

// Interseção de retas ab e cd (assume não paralelas)
Point line_intersect(Point a, Point b, Point c, Point d) {
    Point v1 = b - a, v2 = d - c;
    ld t = (c - a).cross(v2) / v1.cross(v2);
    return a + v1 * t;
}

```

```
// Segmentos ab e cd se intersectam?
bool seg_intersect(Point a, Point b, Point c, Point d)
{
    ld o1 = orient(a, b, c), o2 = orient(a, b, d);
    ld o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 == 0 && o2 == 0 && o3 == 0 && o4 == 0) {
        if (b < a) swap(a, b); if (d < c) swap(c, d);
        return !(b < c || d < a);
    }
    return ((o1 > EPS) != (o2 > EPS)) && ((o3 > EPS) !=
        (o4 > EPS));
}
```

13.2 Fecho convexo

```
// === Andrew's Monotone Chain ===
// Complexidade:  $O(N \cdot \log N)$ 
// Retorna pontos do casco em ordem CCW, sem pontos
// colineares na borda

vector<Point> convex_hull(vector<Point> pts) {
    if (pts.size() <= 1) return pts;
    sort(pts.begin(), pts.end());
    vector<Point> hull(pts.size() * 2);
    int k = 0;

    // Lower hull
    for (int i = 0; i < pts.size(); i++) {
        while (k >= 2 && orient(hull[k-2], hull[k-1],
            pts[i]) <= EPS) k--;
        hull[k++] = pts[i];
    }

    // Upper hull
    for (int i = pts.size()-2, t = k+1; i >= 0; i--) {
        while (k >= t && orient(hull[k-2], hull[k-1],
            pts[i]) <= EPS) k--;
        hull[k++] = pts[i];
    }

    hull.resize(k-1);
    return hull;
}

// Área do polígono (pontos em ordem CCW ou CW)
// Complexidade:  $O(N)$ 
ld polygon_area(vector<Point> &poly) {
    ld area = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++)
        area += poly[i].cross(poly[(i+1) % n]);
    return fabs(area) / 2.0;
}

// Perímetro do polígono
// Complexidade:  $O(N)$ 
ld polygon_perimeter(vector<Point> &poly) {
    ld per = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++)
        per += dist(poly[i], poly[(i+1) % n]);
    return per;
}

// Ponto dentro do polígono? (assume polígono simples)
// 0 = fora, 1 = dentro, 2 = na borda
// Complexidade:  $O(N)$ 
int point_in_polygon(vector<Point> &poly, Point p) {
    int n = poly.size(), wn = 0;
    for (int i = 0; i < n; i++) {
        Point a = poly[i], b = poly[(i+1) % n];
        if (dist_to_seg(a, b, p) < EPS) return 2; // na
            borda
        if (a.y <= p.y + EPS) {
            if (b.y > p.y + EPS && orient(a, b, p) >
                EPS) wn++;
        } else {
            if (b.y <= p.y + EPS && orient(a, b, p) < -
                EPS) wn--;
        }
    }
}
```

```
return wn != 0;
}
```

14 Algoritmos de Ordenação

14.1 Insertion Sort

```
//  $O(N^2)$ 
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

14.2 Merge Sort

```
//  $O(N \cdot \log N)$ 
void merge(vector<int>& vet, int left, int mid, int
    right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (vet[i] <= vet[j])
            temp[k++] = vet[i++];
        else
            temp[k++] = vet[j++];
    }
    while (i <= mid)
        temp[k++] = vet[i++];
    while (j <= right)
        temp[k++] = vet[j++];
    for (i = left, k = 0; i <= right; i++, k++)
        vet[i] = temp[k];
}

// Chamada => merge_sort(vet, 0, vet.size() - 1);
void merge_sort(vector<int>& vet, int left, int right)
{
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(vet, left, mid);
        merge_sort(vet, mid + 1, right);
        merge(vet, left, mid, right);
    }
}
```

14.3 Quicksort

```
//  $O(N \cdot \log N)$ 
int partition(vector<int>& vet, int left, int right) {
    int mid = left + (right - left) / 2;
    int pivot = vet[mid];
    swap(vet[mid], vet[right]);
    int i = left - 1;
    for (int j = left; j < right; j++) {
        if (vet[j] <= pivot) {
            i++;
            swap(vet[i], vet[j]);
        }
    }
    swap(vet[i + 1], vet[right]);
    return i + 1;
}

// Chamada => quick_sort(v, 0, v.size() - 1);
void quick_sort(vector<int>& vet, int left, int right)
{
    if (left < right) {
        int pivot = partition(vet, left, right);
        quick_sort(vet, left, pivot - 1);
        quick_sort(vet, pivot + 1, right);
    }
}
```

14.4 Counting Sort

```
// O(N + K), K = max_val
void counting_sort(vector<int>& vet, int max_val) {
    vector<int> count(max_val + 1, 0);
    for (int x : vet) count[x]++;
    int idx = 0;
    for (int i = 0; i <= max_val; i++)
        while (count[i]--) vet[idx++] = i;
}
```

15 Problemas Especiais

15.1 Torre de Hanoi

```
// Torre de Hanoi: move n discos de 'a' para 'b' usando
'c' como auxiliar
// Complexidade: O(2^n), gera exatamente 2^n - 1
movimentos
vector<pair<int, int>> moves;
void hanoi(int n, int a, int b, int c) {
    if (n == 1) moves.push_back({a, b});
    else {
        hanoi(n - 1, a, c, b);
        hanoi(1, a, b, c);
        hanoi(n - 1, c, b, a);
    }
}
// Uso: hanoi(n, 1, 3, 2);
```


Papel milimetrado para rascunho



